Documentación GSX Práctica 1

Sergi Llobet
Iván García

Índice

Week 0 ..... **3**
WEEK 1 ..... **4**
Preguntas: 7
WEEK 2 ..... **8**
GESTIÓN DEL SERVICIO WEB (NGINX) 8
Preguntas: 10
WEEK 3 ..... **11**
Scripting de Diagnóstico de Procesos 11
Demostración de Control y Señales 11
Límites de Recursos mediante Cgroups 11
Observabilidad y Estrés 12
Preguntas: 12
WEEK 4 ..... **13**
Estructura de Usuarios y Grupos 13
Decisiones de Diseño y Justificación 14
Configuración de Directorios Compartidos 14
Decisiones de Diseño y Justificación 14
Límites de Recursos por Usuario (PAM) 14
Configuración en /etc/security/limits.conf 14
Decisiones de Diseño y Justificación 15
Personalización del Entorno Corporativo 15
Decisiones de Diseño y Justificación 15
Preguntas: 15
Week 5 ..... **17**
Estrategia de Backup y Automatización 20
Procedimientos de Recuperación (Disaster Recovery Runbook) 20
Preguntas: 21
Week 6 ..... **22**
Manual de Operaciones (Operations Runbook) 22
Arquitectura del Sistema 24
Preguntas de reflexión (Ivan) 25
Preguntas de reflexión (Sergi) 26

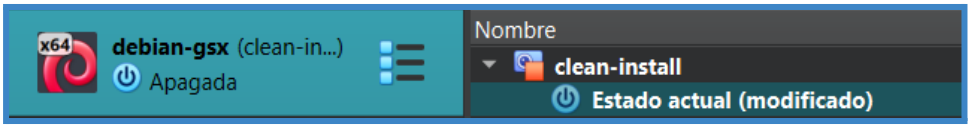
Week 0

Al inicio hemos instalado la imagen de Debian 13 (https://www.debian.org/distrib/, *iso netinst para PC de 64 bits*) en nuestros ordenadores.

Y creamos la máquina Virtual en VirtualBox tal y como especifica el enunciado:

- Nombre del sistema: debian-gsx
- Usuario principal: gsx
- Hostname: debian-gsx
- Dominio: gsx.virtualbox.org
- Recursos hardware: 2 GB de memoria RAM y 1 CPU.
- Almacenamiento: Disco duro de 20 GB con reserva dinámica.

Junto con un backup de la imagen limpia con el nombre “clean-install” para una fácil recuperación del estado. Junto con las especificaciones de recursos y uso de la red NAT.



WEEK 1

En esta semana hemos hemos iniciado el primer script de configuración básico para configurar de manera automática y idempotente el entorno dentro del cual están los siguientes apartados:

- Instalar los paquetes básicos para cualquier montaje de sistemas:
    - **sudo**: Es la herramienta de escalada de privilegios.
    - **kbd**: Son las utilidades del teclado para la consola.
    - **net-tools**: Es el conjunto de herramientas de red clásicas. (`ifconfig` y `netstat`).
    - **openssh-server**: Es el servicio de acceso remoto.
    - **git**: Es el sistema de control de versiones.
    - **tar**: Empaquetar información.
    - **gnupg2**: Encriptar información para más seguridad.
- Reconfiguración del teclado para trabajar con una versión española.
- Desactivación de la GUI de la MV Debian.
- Configurar los permisos sudo a un usuario marcado por parámetro de entrada.
- Hacer un Reboot para aplicar los cambios.

A parte del script hemos decidido como grupo en trabajar con un repositorio git (https://github.com/IvanGP7/GSX-2026) para hacer más cómodo el acceso a los scripts que vayamos desarrollando, a parte de eso cada uno tendrá su rama de trabajo (ivan y sergi) la cual se usará para cambios periódicos y cuando se tenga una versión testeada y estable se irá añadiendo a la rama main.

Una vez realizado eso hemos creado diferentes scripts para la administración:

1. Script de Configuración Inicial (setup_admin.sh)
   Este es el script encargado de crear la estructura de trabajo. Su función es instalar automáticamente todas las herramientas necesarias (como el sistema de copias de seguridad, el control de versiones Git y las utilidades de red) y crear la estructura de carpetas administrativa (**/admin**):
   - **/scripts**: Donde el usuario guardará sus scripts.
   - **/logs**: Donde el usuario podrá redirigir sus salidas para facilitar su trabajo.
   - **/config**: Carpeta de configuración donde se almacenarán scripts e información para mantener la estabilidad del sistema.
   - **/backups**: Carpeta donde se guardarán copias de información empaquetadas.

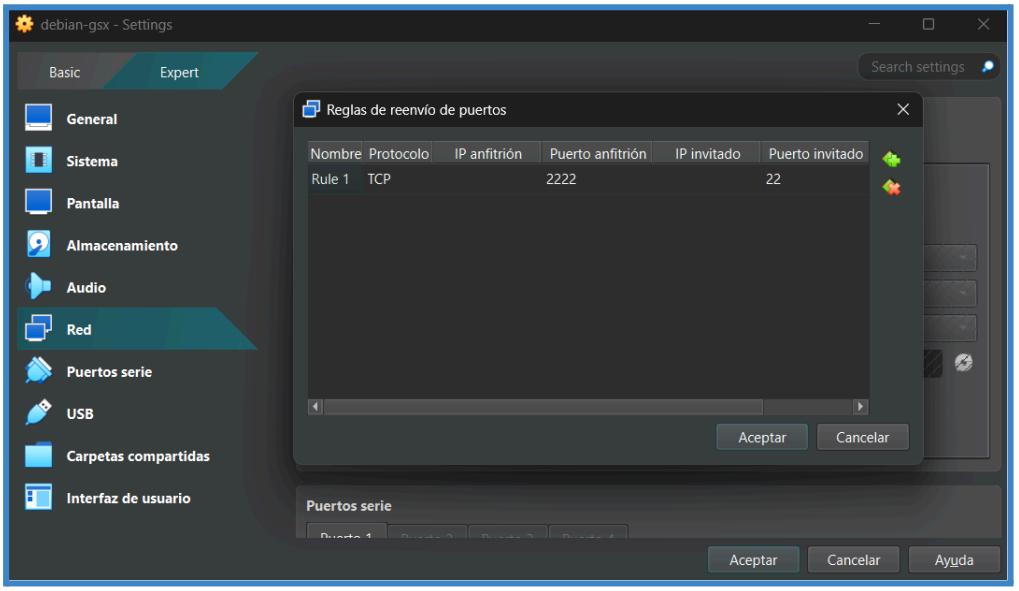
2. Script de Verificación e Integridad (verify_setup.sh)
   Este script supervisa el sistema. Su misión es revisar que todo lo configurado anteriormente siga en su sitio y funcionando correctamente. Comprueba que no se hayan borrado carpetas críticas y que los servicios sigan activos.

3. Script de Copia de Seguridad Protegida (backup.sh)
   Es la herramienta dedicada a la protección de los datos. Se encarga de empaquetar los archivos importantes del directorio administrativo en un solo archivo comprimido,

asegurándose de que se mantengan todos los permisos y atributos de los archivos originales.

Una vez creado todo para una mejor verificación y conexión con la máquina virtual vamos hacer una conexión ssh desde nuestro host. Para esto vamos a tener que configurar 3 cosas.

1. **VirtualBox**: Para que la red de la máquina permita conexiónes ssh estando en nat tendremos que habilitar una rega de reenvío de puertos. Dentro de la máquina le damos "*configuración*", una vez dentro nos movemos al apartado de red que debe estar en NAT y entramos donde ponde "*Reenvío de puertos*" y creamos la conexión con puerto anfitrión 2222 y puerto invitado 22 dejando el resto tal y como está. Tal y como aparece en la siguiente imágen.



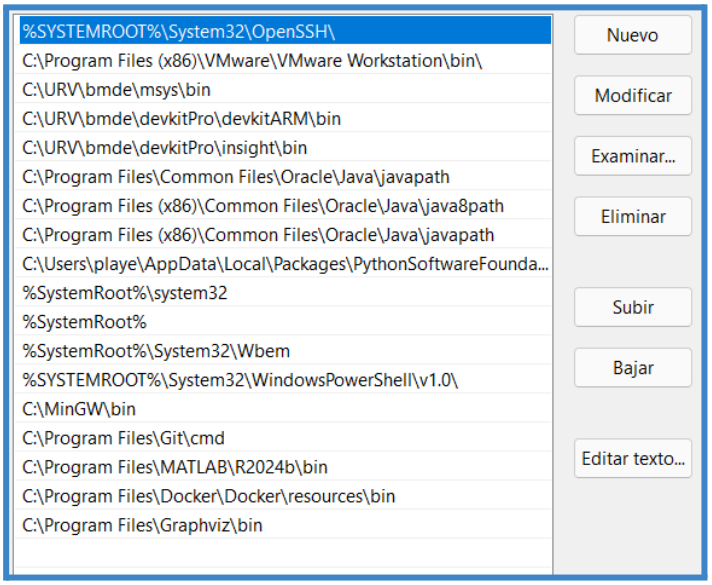
2. **Máquina Virtual**: Dentro del Debian tendremos que asegurarnos que el servicio ssh esté activo usando los comandos "*systemctl*" y debe estar activo.

```
gsx@vbox:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-02-12 16:25:37 CET; 2h 13min ago
 Invocation: e65caa2fecfc4858beee716c5954b6a4
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 843 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 858 (sshd)
      Tasks: 1 (limit: 9449)
     Memory: 8.9M (peak: 27.6M)
        CPU: 197ms
     CGroup: /system.slice/ssh.service
             └─858 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

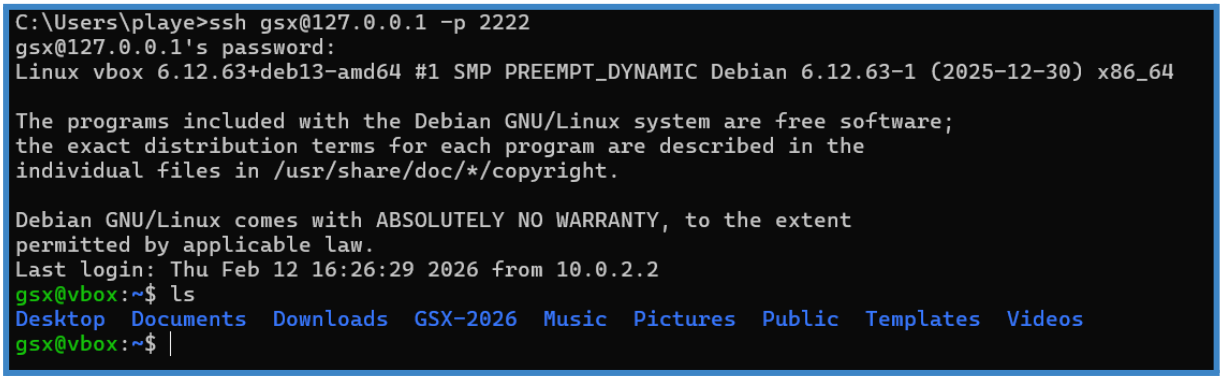
3. **Máquina Host**: Este es el paso que más dolores de cabeza nos ha dado, por defecto windows tiene un paquete ssh default antiguo o la variable de entorno del servicio no está bien configurada en cualquier caso hay que hacer 2 pasos:
    a. Instalar una versión actual:

```
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
```

b. Subir la siguiente línea en la Variable del sistema “*Path*”:



Una vez generado todo podemos ejecutar el comando “ssh gsx@127.0.0.1 -p 2222” en la consola de la terminal y con la contraseña elegida previamente podemos acceder a la máquina virtual sin ningún problema y ejecutar comandos de la misma.



Preguntas:

Seguridad en el acceso: Contraseñas frente a Llaves SSH
El uso de contraseñas es intrínsecamente más arriesgado debido a su vulnerabilidad ante ataques de fuerza bruta y la necesidad de gestionarlas manualmente. Por el contrario, las llaves SSH emplean criptografía asimétrica, lo que elimina el envío de credenciales por la red y permite una autenticación mucho más robusta y automatizable mediante el uso de una clave privada protegida.

Capacidad de restauración ante una reinstalación
Aunque los scripts actuales automatizan con éxito la instalación de herramientas y la creación de la estructura administrativa, no cubren la restauración de los datos personales de los usuarios ni las configuraciones de red específicas fuera del estándar. Para una recuperación total, sería necesario complementar estos procesos con la restauración de los backups encriptados y la configuración manual de secretos, como las propias llaves privadas de acceso.

Prevención de conflictos en la ejecución por el equipo
Para evitar que dos administradores lancen procesos de configuración simultáneos que puedan corromper el sistema, se debe implementar un mecanismo de bloqueo mediante un "archivo lock". Este sistema garantiza que, si un script ya está en ejecución, cualquier otro intento de inicio se detenga inmediatamente con un aviso, asegurando que las tareas de administración se realicen de manera secuencial y segura.

Criterios de almacenamiento: Git frente al Servidor
El repositorio Git se utiliza para centralizar el código fuente, los scripts y la documentación, permitiendo el control de versiones y el trabajo colaborativo entre los ingenieros. Sin embargo, los datos sensibles como claves privadas, contraseñas reales y los propios archivos de backup deben residir exclusivamente en el servidor o en almacenes de secretos seguros, evitando así que información crítica pueda filtrarse a través del historial de cambios del repositorio.

WEEK 2

GESTIÓN DEL SERVICIO WEB (NGINX)

El despliegue de Nginx se ha diseñado bajo el estándar de Systemd para asegurar su disponibilidad continua y facilitar su diagnóstico mediante herramientas estándar del sistema.
Decisiones de Diseño y Justificación

- **Persistencia y Disponibilidad:** Se ha configurado el servicio para que se inicie automáticamente durante el arranque del sistema (boot) y se recupere ante fallos críticos.
- **Política de Reinicio:** Se ha añadido la directiva "Restart=on-failure" con un "RestartSec=5s". Esta decisión garantiza que, en caso de un cierre inesperado (crash), el sistema intente levantar el servicio de nuevo sin intervención manual, reduciendo el tiempo de inactividad (downtime).
- **Visibilidad:** Al integrarse con Systemd, el servicio es totalmente rastreable mediante "systemctl status nginx" y "journalctl -u nginx", permitiendo una auditoría clara de sus procesos y señales de ciclo de vida.

Automatización de backups mediante systemd

En lugar de utilizar el planificador Cron, se ha implementado un sistema compuesto por una unidad de servicio y una unidad de temporizador (timer).
Decisiones de Diseño y Justificación

- **Estructura Desacoplada:** El archivo "backup.service" define el "qué" (ejecución del script de la Semana 1), mientras que "backup.timer" define el "cuándo".
- **Ventajas sobre Cron:** Se ha elegido Systemd Timers porque permite una integración nativa con el resto del sistema. Esto asegura que si el servidor está apagado en el momento programado, el backup se ejecute inmediatamente al encenderlo gracias a la directiva "Persistent=true". Además, toda la salida (stdout/stderr) del script se captura automáticamente en los logs estructurados del sistema.

Logrotate

Para evitar el agotamiento del espacio en disco y mantener una observabilidad clara, se ha diseñado un sistema de rotación de logs independiente para la carpeta "/admin/logs/".

Configuración del Archivo .conf

La configuración aplicada en "/admin/config/custom_logs.conf" es la siguiente:
"/admin/logs/*.log {

- su
- root
- gsx
- daily
- rotate 7
- compress
- delaycompress

- missingok
- notifempty
- olddir /admin/logs/backups
- create 0640 gsx gsx }"

Decisiones de Diseño y Justificación

- **Independencia por Comodín:** Al usar el patrón "*.log", Logrotate trata cada archivo con nombre distinto como un ciclo de rotación independiente. Esto permite gestionar logs por usuario o por servicio de forma aislada.
- **Seguridad (su root gsx):** Es una decisión crítica de diseño. Dado que el directorio puede tener permisos específicos, esta directiva permite que el proceso Logrotate (root) actúe con los privilegios del usuario dueño de los logs, evitando errores de permisos "insecure permissions".
- **Eficiencia de Espacio:** Se utiliza "daily" y "rotate 7" para mantener una ventana de una semana de historial. La combinación de "compress" y "delaycompress" asegura que los logs antiguos no ocupen espacio innecesario sin interferir con procesos que aún tengan el archivo abierto.

3. Observabilidad y diagnóstico

La capacidad de diagnóstico se ha centrado en la centralización de logs en Journald y la facilidad de consulta.

Decisiones de Diseño y Justificación

- **Centralización:** Todos los scripts de administración redirigen sus errores a Journald. Esto permite al administrador filtrar eventos por prioridad o tiempo de forma unificada.
- **Evidencia de Fallos:** El sistema permite diagnosticar fallos mediante la correlación de eventos. Por ejemplo, si un backup falla, el administrador puede consultar el log histórico para ver el código de salida del script y los errores de sistema concurrentes.

Preguntas:

¿Qué pasa si Nginx cae a las 3 AM?
Si has configurado un Systemd Timer o un Cron job de monitoreo, el script detectará el fallo en la siguiente ejecución y reiniciará el servicio automáticamente. El administrador se enterará revisando los logs del sistema o mediante una alerta configurada que notifique cuando el estado del servicio cambie de active a failed.

¿Cómo pruebas que un servicio reinicia automáticamente?
La forma más directa es simular un fallo real terminando el proceso de forma manual mediante el comando sudo systemctl stop nginx o, de forma más agresiva, matando su PID con kill -9. Inmediatamente después, debes ejecutar tu script de monitoreo o esperar al evento de Systemd para verificar con Systemctl status si el servicio ha vuelto a la vida por sí solo.

¿Cómo detectar fallos silenciosos en backups y qué métricas importan?
Para evitar fallos silenciosos, debes monitorizar el código de salida (exit status) del script de backup y el tamaño del archivo resultante; un backup de 0 bytes es un fallo claro. Las métricas críticas son la fecha de la última modificación exitosa, el espacio disponible en el destino (df -h) y la integridad del archivo mediante sumas de verificación (MD5/SHA).

¿Cómo explicarías un fallo al equipo usando solo los logs?
Identificariamos la marca de tiempo (timestamp) exacta del error en /var/log/nginx/error.log o el log personalizado en /admin/logs/ para correlacionar con eventos del sistema. Mostraría la línea específica del log que indica el motivo técnico (por ejemplo, "Connection refused" o "Out of memory") para explicar de forma objetiva qué causó la interrupción y cuánto tiempo duró el downtime.

WEEK 3

Scripting de Diagnóstico de Procesos

Se ha implementado un script maestro de diagnóstico (process_diag.sh) diseñado bajo el principio de idempotencia para asegurar que el entorno de auditoría esté siempre disponible y funcional.

Decisiones de Diseño y Justificación

- **Idempotencia de Entorno:** El script verifica automáticamente la existencia de dependencias críticas como pstree o htop. Si no se encuentran, las instala sin intervención manual, garantizando que las pruebas de diagnóstico no fallen por falta de herramientas.
- **Visibilidad Jerárquica:** Se ha integrado el uso de pstree -pnhu para mostrar no solo los procesos aislados, sino sus relaciones de parentesco. Esta decisión permite al administrador identificar rápidamente qué servicio (padre) ha generado una carga de trabajo anómala (hijo).
- **Extracción de Métricas Críticas:** El script filtra la salida de ps mediante awk y column -t para presentar únicamente los datos vitales: PID, %CPU, %MEM, VSZ (Memoria Virtual) y RSS (Residente). Esto facilita una lectura rápida en situaciones de estrés del servidor.

Demostración de Control y Señales

Para validar la capacidad de respuesta del sistema ante incidentes, se ha desarrollado un entorno de pruebas con un "workload script" que utiliza el comando yes en segundo plano.

Decisiones de Diseño y Justificación

- Manejo de Señales (Traps): El script captura activamente señales como SIGINT, SIGUSR1 y SIGUSR2. Esta técnica de programación permite que los procesos tengan una "salida elegante" (Graceful Stop), limpiando sus propios subprocesos antes de finalizar, en lugar de dejar procesos huérfanos.
- Diferenciación de Parada (SIGTERM vs SIGKILL): Se ha documentado la diferencia crítica entre una terminación educada (kill -15) y una forzosa (kill -9). El diseño del script demuestra que, bajo un SIGKILL, el sistema operativo detiene el proceso de forma inmediata, impidiendo cualquier tarea de limpieza programada.

Límites de Recursos mediante Cgroups

Se ha implementado una política de aislamiento de recursos para el servicio Nginx utilizando las capacidades nativas de Systemd y cgroups v2.

Configuración del Override (nginx.service.d/override.conf)
La configuración aplicada mediante systemctl edit nginx es la siguiente:

```
[Service]
CPUAccounting=yes
MemoryAccounting=yes
CPUQuota=10%
MemoryMax=50M
MemoryHigh=40M
```

Decisiones de Diseño y Justificación

- **Aislamiento de CPU (CPUQuota=10%):** Es una decisión de seguridad crítica. Al limitar a Nginx al 10% de un núcleo, se garantiza que un ataque de denegación de servicio (DoS) o un error de bucle infinito no sature la CPU global, permitiendo que el administrador mantenga el acceso SSH al servidor.
- **Gestión de Memoria (MemoryMax/High):** Se ha establecido un límite estricto de 50MB. El uso de MemoryHigh a 40MB actúa como una advertencia para el sistema, permitiendo que el kernel empiece a liberar caché antes de llegar al límite fatal de MemoryMax que dispararía el OOM Killer.
- **Evidencia de Cumplimiento:** La efectividad de estos límites se ha validado mediante systemctl show y la monitorización en tiempo real con systemd-cgtop, demostrando que los límites se aplican a nivel de kernel de forma transparente para el servicio.

Observabilidad y Estrés

La validación del sistema se ha completado forzando condiciones de carga para observar el comportamiento de los límites impuestos.

Decisiones de Diseño y Justificación

- **Validación Activa:** Se ha utilizado apache2-utils (ab) para estresar el servidor web. Esta prueba proporciona la evidencia necesaria de que, bajo carga extrema, el consumo de recursos de Nginx se "clava" en los umbrales definidos sin afectar al resto del sistema operativo.
- **Diagnóstico de Fallos de Memoria:** Se ha verificado que, al superar los límites de memoria, el sistema genera trazas claras en dmesg y journalctl, permitiendo una auditoría post-mortem de por qué un proceso fue finalizado por el sistema.

Preguntas:

¿En qué se diferencia matar un proceso con SIGTERM de hacerlo con SIGKILL? ¿Cuándo usarías cada uno?

SIGTERM (Señal 15) es una solicitud "educada" que se envía a un proceso para que se detenga. Le permite ejecutar rutinas de limpieza (Graceful Stop), como cerrar conexiones de red o guardar archivos antes de salir. Por el contrario, SIGKILL (Señal 9) es una orden

directa a nivel de kernel que destruye el proceso instantáneamente, sin darle oportunidad de guardar su estado. Siempre se debe usar SIGTERM por defecto (kill <PID>), reservando SIGKILL (kill -9 <PID>) exclusivamente como último recurso cuando un proceso se ha bloqueado completamente, no responde o se ha convertido en un proceso "zombie".
Si tu servicio recibe una señal, ¿cómo debe responder? ¿Debe guardar su estado antes de salir?

Un servicio bien diseñado debe capturar las señales de terminación (utilizando comandos como trap en nuestros scripts o manejadores nativos en el código). Al recibir una señal como SIGTERM, el servicio debe interceptarla, pausar la aceptación de nuevas peticiones, guardar cualquier estado o dato en memoria hacia el disco (para evitar corrupción de datos) y, finalmente, cerrarse devolviendo un código de salida 0 (éxito) al sistema operativo.

¿Cómo verificas que un límite de recursos está funcionando realmente?

Para verificar la efectividad de los límites (ya sean mediante cgroups o limits.conf), la técnica estándar es crear un "Workload Script" (un script de estrés) diseñado específicamente para fallar. Por ejemplo, para probar un límite de memoria, se programa un script que intente almacenar datos infinitos en la RAM. Si el límite está bien configurado, el sistema operativo (a través del OOM Killer) intervendrá y "matará" el proceso al alcanzar el umbral máximo, registrando el evento en dmesg o journalctl. Esto proporciona una evidencia empírica de que el entorno es seguro.

Si el trabajo de un desarrollador está consumiendo el 90% de la CPU, ¿es un problema? ¿Cómo lo decides?

El simple hecho de usar el 90% de la CPU no es un problema por sí solo; de hecho, si es un proceso legítimo (como la compilación de un proyecto pesado), significa que el sistema está rindiendo al máximo. Se considera un "problema" únicamente bajo dos criterios:

1. **Inanición (Starvation):** Si ese proceso impide que servicios críticos del sistema (como el servidor web Nginx o el acceso por SSH) respondan a tiempo.
2. **Naturaleza del proceso:** Si, tras auditarlo con top o htop, determinamos que el proceso es un bucle infinito causado por un error en el código. La solución no es siempre matar el proceso, sino gestionarlo cambiando su prioridad con comandos como renice para que ceda ciclos de CPU a los servicios críticos.

WEEK 4

Estructura de Usuarios y Grupos

Se ha establecido una jerarquía basada en roles para el equipo "GreenDevCorp", garantizando la independencia de procesos y la privacidad de datos.

Decisiones de Diseño y Justificación

- Segregación por Grupos (greendevcorp): Se ha creado un grupo común como identidad primaria para los usuarios dev1 a dev4. Esta decisión facilita la gestión de permisos masivos y asegura que cualquier archivo nuevo pertenezca automáticamente al colectivo del proyecto.
- Privacidad de Directorios Home: Cada usuario cuenta con un entorno /home aislado. Siguiendo el estándar de seguridad, los permisos se han configurado para impedir que un desarrollador pueda leer o modificar los archivos personales de otro, mitigando riesgos de fuga de información interna.
- Idempotencia en el Despliegue: El script de creación utiliza verificaciones mediante getent e id. Esto permite ejecutar el mantenimiento de la infraestructura múltiples veces sin generar errores por duplicidad, manteniendo el sistema en el estado deseado.

Configuración de Directorios Compartidos

Se ha diseñado un espacio de colaboración técnica en /home/greendevcorp/ utilizando bits de permisos especiales para gestionar la concurrencia.

Decisiones de Diseño y Justificación

- Uso de Setgid en shared/: Se ha aplicado el permiso chmod 2770. Esta es una decisión crítica: garantiza que todo archivo creado en la carpeta compartida herede el grupo greendevcorp, permitiendo que el resto del equipo pueda colaborar sin necesidad de cambiar permisos manualmente cada vez.
- Implementación de Sticky Bit (T): Para evitar el borrado accidental o malintencionado, se ha activado el Sticky Bit. Esto asegura que solo el dueño de un archivo (o root) pueda eliminarlo, incluso si otros tienen permisos de escritura en la carpeta.
- Control de Escritura en done.log: Se ha asignado la propiedad a dev1 con permisos 644. Esto cumple el requisito de negocio: todos pueden auditar las tareas completadas (lectura), pero solo el responsable autorizado puede registrar nuevos hitos (escritura).

Límites de Recursos por Usuario (PAM)

A diferencia de los límites de servicios (Semana 3), aquí se han aplicado restricciones a nivel de sesión de usuario mediante pam_limits.so.

Configuración en /etc/security/limits.conf

```
@greendevcorp    hard    nproc           50
@greendevcorp    hard    nofile          1024
@greendevcorp    hard    as              524288
@greendevcorp    soft    cpu             1
```

Decisiones de Diseño y Justificación

- Prevención de Fork Bombs (nproc 50): Al limitar el número de procesos simultáneos, se protege al servidor de scripts mal programados o ataques que intenten colapsar la tabla de procesos del kernel.
- Restricción de Memoria Virtual (as 512MB): Se ha validado mediante el error MemoryError en Python. Esta medida evita que un único usuario monopolice la memoria RAM, garantizando que el servidor web (Nginx) y otros servicios críticos siempre tengan recursos disponibles.
- Persistencia de Configuración: Los límites se han aplicado al grupo (@) y no a individuos, facilitando la escalabilidad del equipo (RBAC - Role Based Access Control).

Personalización del Entorno Corporativo

Se ha estandarizado la experiencia de la shell mediante scripts globales en /etc/profile.d/.

Decisiones de Diseño y Justificación

- Inyección en el PATH: Se ha automatizado la inclusión de /home/greendevcorp/bin en la variable de entorno de los desarrolladores. Esto permite la ejecución de herramientas del equipo de forma directa, eliminando la necesidad de rutas absolutas y mejorando la productividad.
- Estandarización vía Alaliases: Se han definido alias como ll (listado largo) y logs (lectura del registro de tareas). Esta decisión reduce la curva de aprendizaje para nuevos miembros y unifica los comandos de diagnóstico del equipo.
- Auditoría de Seguridad: Se ha implementado un script de verificación que simula accesos no autorizados y desbordamientos de recursos. Las evidencias (capturas de pantalla) confirman que el sistema bloquea activamente cualquier acción que supere los límites de Mínimo Privilegio.

Preguntas:

Si un archivo está en un directorio compartido, es propiedad de dev1, pero necesita ser legible por todos los miembros del equipo, ¿qué permisos establecerías? ¿Por qué?

Si el directorio padre tiene configurado el bit **Setgid** (como es nuestro caso), el archivo heredará automáticamente el grupo greendevcorp al ser creado por dev1. Por lo tanto, bastaría con asignar permisos octales 640 (rw- r-- ---) o 644 (rw- r-- r--). Esto garantiza que dev1 mantiene el control total (lectura y escritura), mientras que el resto del equipo, al pertenecer al grupo greendevcorp, obtiene permisos exclusivos de lectura, cumpliendo estrictamente con el principio de Mínimo Privilegio (Least Privilege).

¿Cuál es la diferencia entre aplicar Setgid en un directorio frente a un archivo? ¿Por qué usarías cada uno?

- **En un directorio:** Fuerza a que todos los archivos y subdirectorios nuevos creados dentro de él hereden el grupo del directorio en lugar del grupo principal del usuario creador. Se utiliza sistemáticamente para crear carpetas de colaboración de equipos (como nuestro /home/greendevcorp/shared).
- **En un archivo (ejecutable):** Hace que, sin importar quién ejecute el programa, este corra con los privilegios del grupo propietario del archivo. Se usaría en herramientas administrativas muy concretas que necesiten acceder a recursos restringidos del sistema, aunque conlleva mayores riesgos de seguridad si el ejecutable tiene vulnerabilidades.

Si configuras mal los permisos y un usuario no puede acceder a un archivo que necesita, ¿cómo realizas el "troubleshooting" (diagnóstico)?

El proceso de diagnóstico sigue una estructura lógica de tres pasos:

1. Utilizar ls -la para revisar los permisos estándar (propietario, grupo y bit de ejecución en toda la ruta de carpetas padre, ya que sin el bit x en una carpeta, no se puede acceder a su contenido).
2. Utilizar getfacl para identificar si existen Listas de Control de Acceso (ACLs POSIX) restrictivas que estén sobrescribiendo los permisos estándar.
3. Emplear el comando su - <usuario> para asumir temporalmente la identidad del desarrollador afectado, ejecutar el comando groups para verificar su membresía, e intentar acceder al archivo para reproducir el error exacto (Permission denied).

¿Cómo verificarías que tu modelo de permisos realmente impone la política de seguridad que pretendías?

La validación se realiza mediante la ejecución de scripts automatizados de auditoría o pruebas de penetración internas. Esto implica crear un script que asuma sistemáticamente el rol de los diferentes usuarios (por ejemplo, dev2 intentando eliminar un archivo de dev1 en una carpeta con *Sticky Bit*). Si el sistema arroja errores de "Permiso Denegado" exactamente en las operaciones no autorizadas, y permite las autorizadas, obtenemos una evidencia auditable de que la arquitectura de control de acceso funciona en un entorno real.

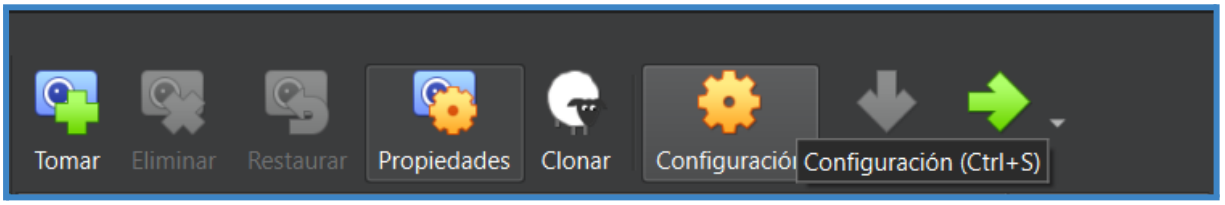
Week 5

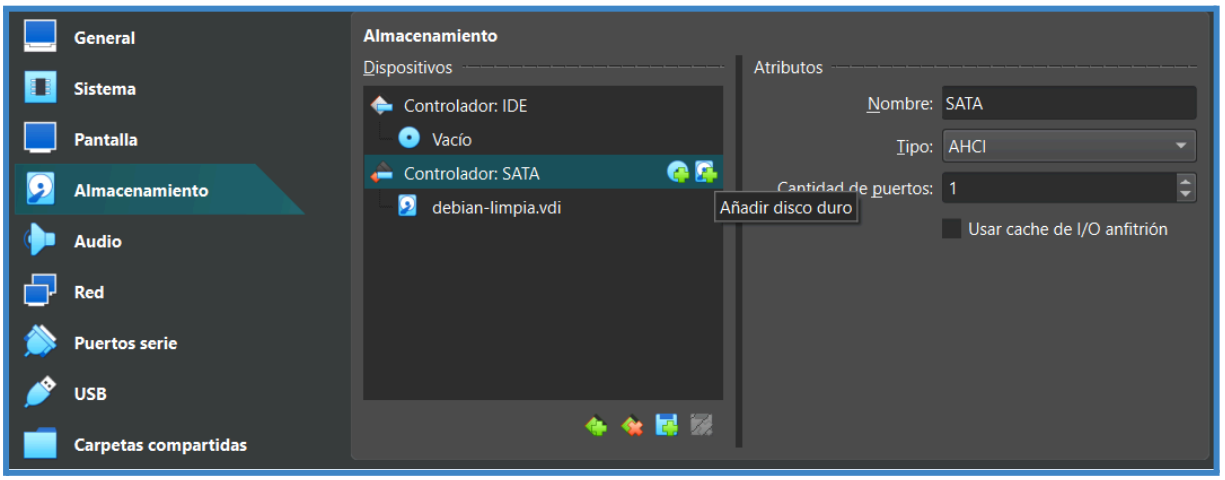
La infraestructura de datos de GreenDevCorp se ha rediseñado para garantizar la persistencia, escalabilidad y disponibilidad de los activos de la startup, aplicando estándares de redundancia y automatización de backups. Esta semana aborda el requerimiento más crítico de cualquier administrador de sistemas: asegurar que la información no se pierda bajo ningún concepto.

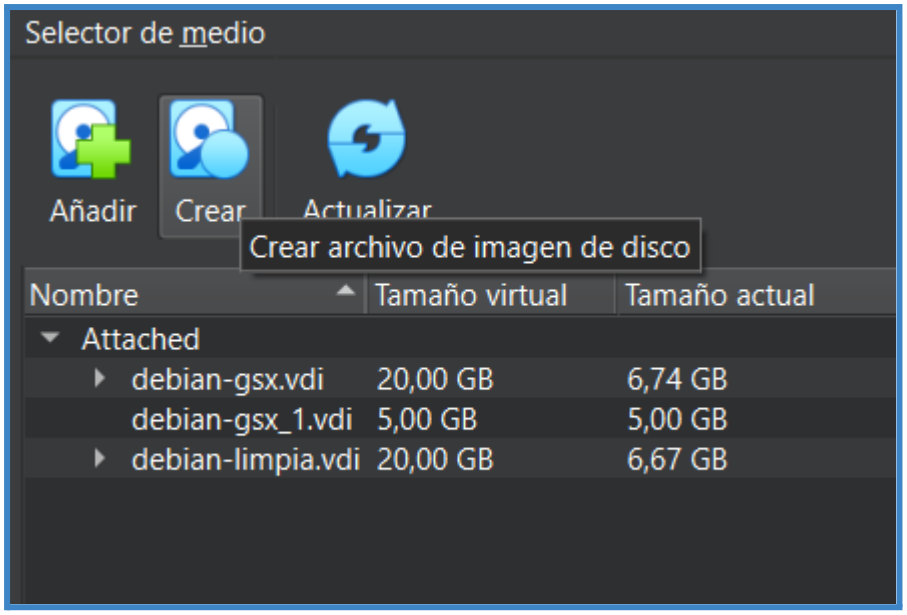
Arquitectura de Almacenamiento (Storage Architecture) y Gestión de Discos

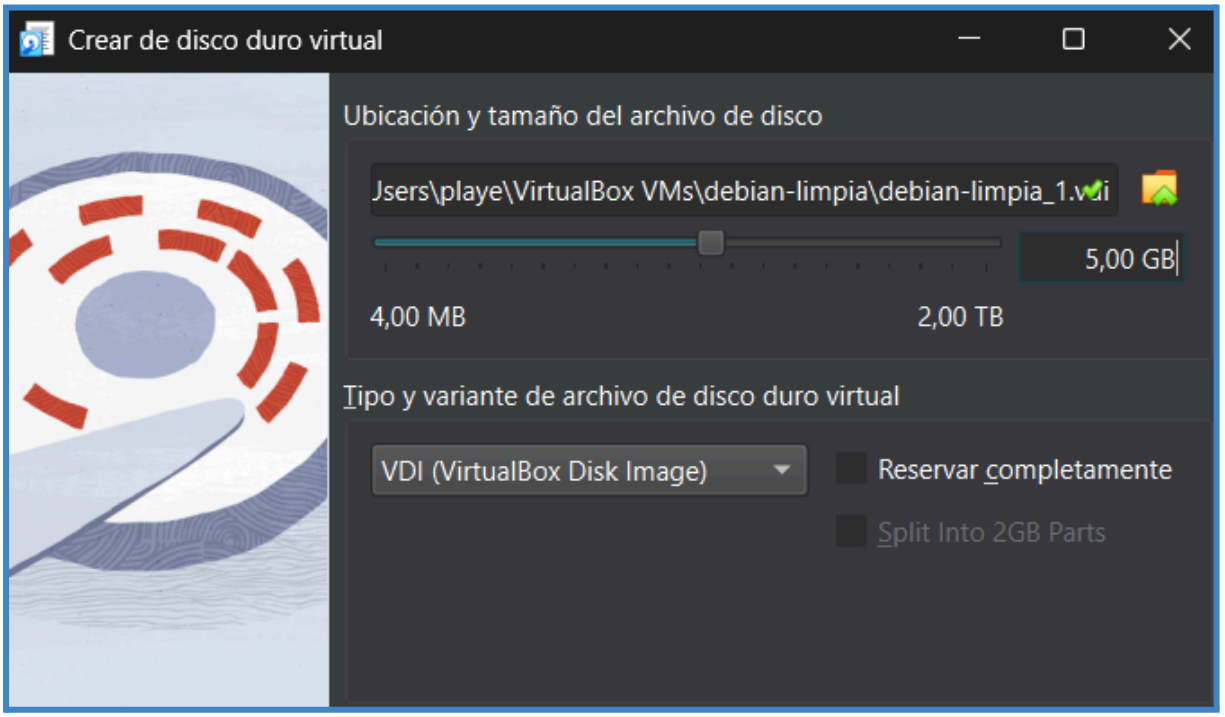
Para cumplir con el principio de separación de responsabilidades y evitar que el crecimiento de los datos sature el sistema operativo, se ha expandido la capacidad del servidor mediante la adición de una unidad de disco secundaria.

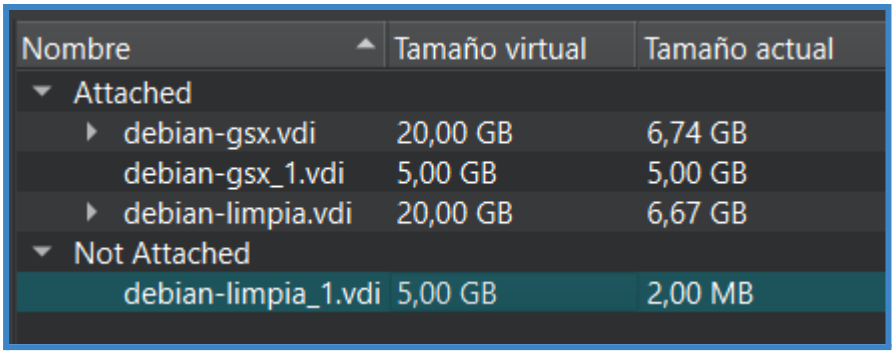
Para realizar esta practica tocará crear un disco extra de datos:

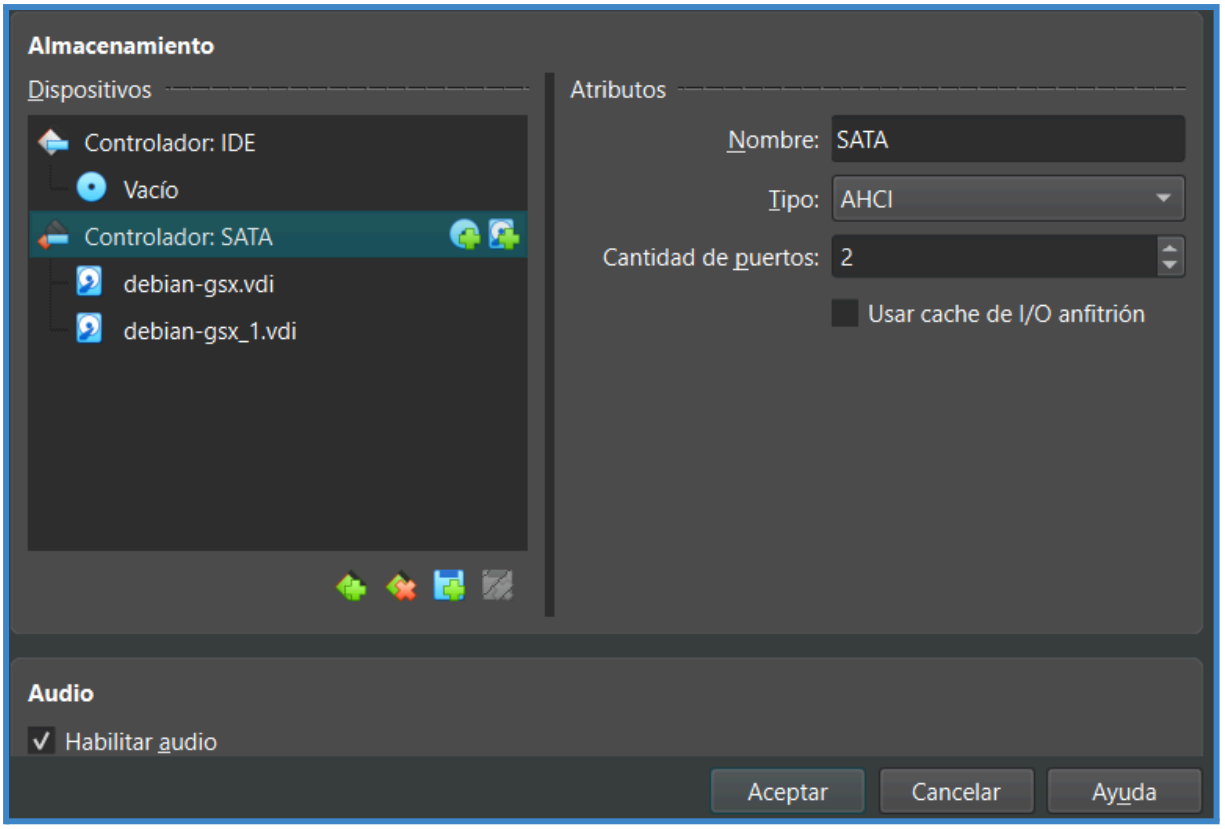





Selector de medio
Añadir
Crear
Actualizar
Crear archivo de imagen de disco
Nombre
Tamaño virtual
Tamaño actual
Attached
debian-gsx.vdi 20,00 GB 6,74 GB
debian-gsx_1.vdi 5,00 GB 5,00 GB
debian-limpia.vdi 20,00 GB 6,67 GB


Crear de disco duro virtual
Ubicación y tamaño del archivo de disco
Jsers\playe\VirtualBox VMs\debian-limpia\debian-limpia_1.vdi
5,00 GB
4,00 MB
2,00 TB
Tipo y variante de archivo de disco duro virtual
VDI (VirtualBox Disk Image)
Reservar completamente
Split Into 2GB Parts


Nombre
Tamaño virtual
Tamaño actual
Attached
debian-gsx.vdi 20,00 GB 6,74 GB
debian-gsx_1.vdi 5,00 GB 5,00 GB
debian-limpia.vdi 20,00 GB 6,67 GB
Not Attached
debian-limpia_1.vdi 5,00 GB 2,00 MB




```
gsx@vbox:~$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   20G  0 disk
├─sda1   8:1    0 18.9G  0 part /
├─sda2   8:2    0    1K  0 part
└─sda5   8:5    0  1.1G  0 part [SWAP]
sdb      8:16   0    5G  0 disk
sr0     11:0    1 1024M  0 rom
```

Decisiones de Diseño y Planificación de Capacidad:

- Aislamiento Físico/Virtual: * Disco Principal (`sda` - 20GB): Reservado para el Sistema Operativo, archivos binarios, logs del sistema y configuraciones de servicios como Nginx.
    - **Disco de Datos (sdb - 5GB):** Nueva unidad dedicada exclusivamente al almacenamiento de copias de seguridad (backups) y, potencialmente, datos compartidos del equipo de GreenDevCorp. Esta separación garantiza que si el disco principal falla o se llena por completo, los backups permanezcan intactos y el sistema no colapse por falta de espacio en la partición raíz `/`.
- Particionado y Sistema de Archivos: Se ha particionado el nuevo disco (`/dev/sdb1`) y se ha formateado utilizando el sistema de archivos `ext4`. La elección de `ext4` se justifica por ser un sistema *journaled* maduro y altamente fiable, ideal para prevenir la corrupción de datos en caso de apagados repentinos.

- Montaje Persistente (FSTAB): El disco se ha montado en la ruta `/mnt/data_storage`. Para garantizar que este montaje sobreviva a los reinicios del servidor, se ha configurado de forma persistente en el archivo `/etc/fstab`.
    - *Decisión Crítica:* En lugar de usar la ruta del dispositivo (`/dev/sdb1`), se ha referenciado utilizando su **UUID** (Identificador Único Universal). Esta decisión técnica evita fallos críticos durante el arranque del sistema operativo si, en el futuro, se añaden más discos y el orden de los dispositivos de hardware cambia.

Estrategia de Backup y Automatización

El diseño de la política de copias de seguridad se ha fundamentado en mitigar los escenarios de fallo más comunes (borrado accidental, corrupción de archivos o fallo del disco principal) siguiendo buenas prácticas de la industria.

Decisiones de Diseño y Especificaciones Técnicas:

- **Principio 3-2-1:** La estrategia implementada sienta las bases para la regla 3-2-1. Al guardar los backups en el disco `/dev/sdb`, nos aseguramos de tener los datos originales en `sda` y una copia en un soporte virtual distinto (`sdb`). *(Nota: Para cumplir el principio al 100%, la tercera copia debería enviarse a un almacenamiento externo o en red).*
- **Estrategia Incremental (rsync):** Se ha descartado el uso de backups "Completos" (Full) diarios mediante `tar` porque, a la larga, consumirían todo el espacio del disco de 5GB. En su lugar, se ha optado por una estrategia **Incremental** utilizando la herramienta `rsync`.
    - *Directiva `--link-dest`:* Esta opción de `rsync` es clave en nuestra arquitectura. Permite crear copias de seguridad diarias que parecen completas (contienen todos los archivos), pero que internamente utilizan "hard links" para los archivos que no han cambiado desde el día anterior. Esto reduce drásticamente el espacio consumido en disco y el tiempo de ejecución.
- **Automatización Nativa (Systemd Timers):** * **Ubicación y Privilegios:** El script de ejecución se ha centralizado en `/admin/config/backup_manager.sh` para restringir su modificación únicamente al personal de operaciones (root/sudo).
    - **Temporizador (`greendev-backup.timer`):** La frecuencia se ha establecido diariamente a las 02:00 AM, aprovechando la ventana de mantenimiento donde hay menos tráfico o actividad de los desarrolladores.
    - *Ventaja sobre Cron:* Se utiliza la directiva `Persistent=true` de Systemd. Si el servidor estuviera apagado a las 02:00 AM, el backup no se "saltará", sino que se ejecutará inmediatamente en el momento en que el servidor vuelva a encenderse.

Procedimientos de Recuperación (Disaster Recovery Runbook)

Un backup no tiene valor si no se puede restaurar. Se ha documentado el procedimiento paso a paso ("Step-by-Step") que el administrador debe seguir en caso de pérdida de datos críticos en la carpeta de trabajo del equipo (`/home/greendevcorp/`):

1. **Identificación del Snapshot:** Localizar el punto de restauración deseado dentro del nuevo disco de backups en `/mnt/data_storage/backups/`. Para facilitar la operación durante una crisis, existe un directorio (o enlace simbólico) llamado `latest` que siempre apunta a la versión válida más reciente.
2. **Restauración de la Información:** Ejecutar el comando de recuperación utilizando `rsync` en modo archivo para volcar los datos desde el disco de backup al disco principal:
   `sudo rsync -av /mnt/data_storage/backups/latest/ /home/greendevcorp/`
3. **Verificación de Integridad y Permisos:** Confirmar mediante `ls -la /home/greendevcorp/` que los archivos restaurados mantienen a los propietarios y grupos originales (dev1, greendevcorp) para que los desarrolladores recuperen su acceso inmediatamente sin errores de permisos.
4. **Auditoría de Logs:** Tras el incidente, es obligatorio revisar los registros en `/var/log/greendev_backup.log` (o en Journald) para confirmar que no se produjeron errores de I/O o de red durante el último ciclo de sincronización antes del desastre.

Preguntas:

¿Cómo reducir el overhead de almacenamiento si haces un backup cada noche y cuál es el "trade-off" (compromiso)?

Para reducir drásticamente el consumo de espacio, se ha descartado hacer backups Completos diarios. En su lugar, se utilizan backups **Incrementales** (aprovechando la función --link-dest de rsync). De esta forma, solo se copian físicamente los archivos que han cambiado, y los demás se referencian mediante *hard links*. El *trade-off* (compromiso) es la complejidad de la restauración: un backup incremental depende de los anteriores para reconstruir el estado completo de los datos. Si un eslabón de la cadena se corrompe, los backups posteriores podrían quedar inservibles.

Si pierdes el servidor principal, el sistema está diseñado para sobrevivir a la pérdida o corrupción de la partición principal (sda). Al almacenar los backups en un disco secundario físicamente separado (o virtualmente separado, sdb), el desastre en el disco de sistema no afecta a las copias. Hemos probado el procedimiento simulando un borrado de /home/greendevcorp/ y ejecutando el comando de restauración desde /mnt/data_storage/backups/latest/, verificando que todos los archivos se restauraron con sus respectivos dueños y permisos intactos.

¿Cuánto tiempo tardaría la recuperación (RTO) y es aceptable para la startup?

El RTO (Recovery Time Objective) de nuestro sistema es mínimo, de apenas un par de minutos. Al tener el disco de backups montado localmente en la máquina (/mnt/data_storage), la velocidad de restauración está limitada únicamente por la velocidad de lectura/escritura del disco duro (I/O). Para una startup ágil de 4 a 10 empleados, este tiempo de recuperación es excelente y supera con creces el RTO de descargar backups gigantes desde un almacenamiento en la nube.

Si una ubicación de backup se corrompe, ¿tienes otra copia? (La importancia del 3-2-1) Actualmente el sistema cuenta con 2 copias locales de los datos (la de producción en sda y el backup en sdb). Si la máquina entera sufre un desastre físico catastrófico o un ataque de ransomware que encripte los discos montados, perderíamos la información. Para cumplir estrictamente el principio **3-2-1** , nuestro siguiente paso evolutivo en la arquitectura sería configurar un *Offsite Backup* (por ejemplo, sincronizar automáticamente esos archivos hacia un servidor remoto vía SSH/SCP o usar un recurso NFS externo de la universidad).

¿Cómo gestionarías el backup de una base de datos que está recibiendo escrituras en ese momento?

Nunca se debe copiar el archivo físico de una base de datos en ejecución (.db o ficheros de MySQL/PostgreSQL) usando herramientas convencionales como tar o cp, ya que si se realiza una transacción en el instante de la copia, el archivo resultante quedará corrupto (inconsistente). La solución profesional es hacer un **Dump** lógico de la base de datos usando las herramientas nativas del motor (por ejemplo, mysqldump o pg_dump) antes de ejecutar el backup de archivos, o bien utilizar *snapshots* a nivel de bloques (como las instantáneas de LVM) que congelan el estado del sistema de archivos en una fracción de segundo.

Week 6

Para garantizar la mantenibilidad a largo plazo de la infraestructura de GreenDevCorp y asegurar una transición fluida a futuros ingenieros de operaciones, se ha elaborado el siguiente Manual de Operaciones y la documentación arquitectónica del sistema.

Manual de Operaciones (Operations Runbook)

Este manual define los procedimientos operativos estándar (SOP) para la gestión diaria de la infraestructura.

1.1. Gestión de Identidades y Accesos (IAM)

- **¿Cómo añadir un nuevo desarrollador al equipo?**
    1. Ejecutar el comando para crear el usuario asignándole el grupo principal del equipo y creando su carpeta /home:
       sudo useradd -m -g greendevcorp -s /bin/bash nuevodev

    2. Asignar una contraseña temporal: sudo passwd nuevodev
    3. Forzar el cambio de contraseña en el primer inicio de sesión: sudo chage -d 0 nuevodev
    4. El usuario heredará automáticamente los límites de recursos de pam_limits y los *aliases* del entorno al pertenecer al grupo greendevcorp.
- **¿Cómo gestionar la salida de un miembro del equipo?**
    1. Bloquear inmediatamente el acceso (caducar cuenta): sudo usermod -E 1 nuevodev
    2. Revocar las sesiones activas: sudo killall -u nuevodev
    3. Copiar cualquier archivo de su directorio personal que sea propiedad de la empresa a /home/greendevcorp/shared/.
    4. Eliminar el usuario y su directorio personal: sudo userdel -r nuevodev

1.2. Mantenimiento y Verificación de Servicios

- **¿Cómo comprobar si los servicios están funcionando correctamente?**
    - Para el servidor web: systemctl status nginx (debe indicar *active (running)*).
    - Para los backups: systemctl list-timers --all (verificar que backup-semana1.timer está activo y planificado).
- **¿Cómo diagnosticar si un servicio ha fallado?**
    - Revisar los logs del demonio mediante Journald: journalctl -u nginx -n 50 --no-pager. Esto mostrará las últimas 50 líneas de registro para identificar códigos de error específicos.

1.3. Diagnóstico de Rendimiento (Troubleshooting de un sistema lento)

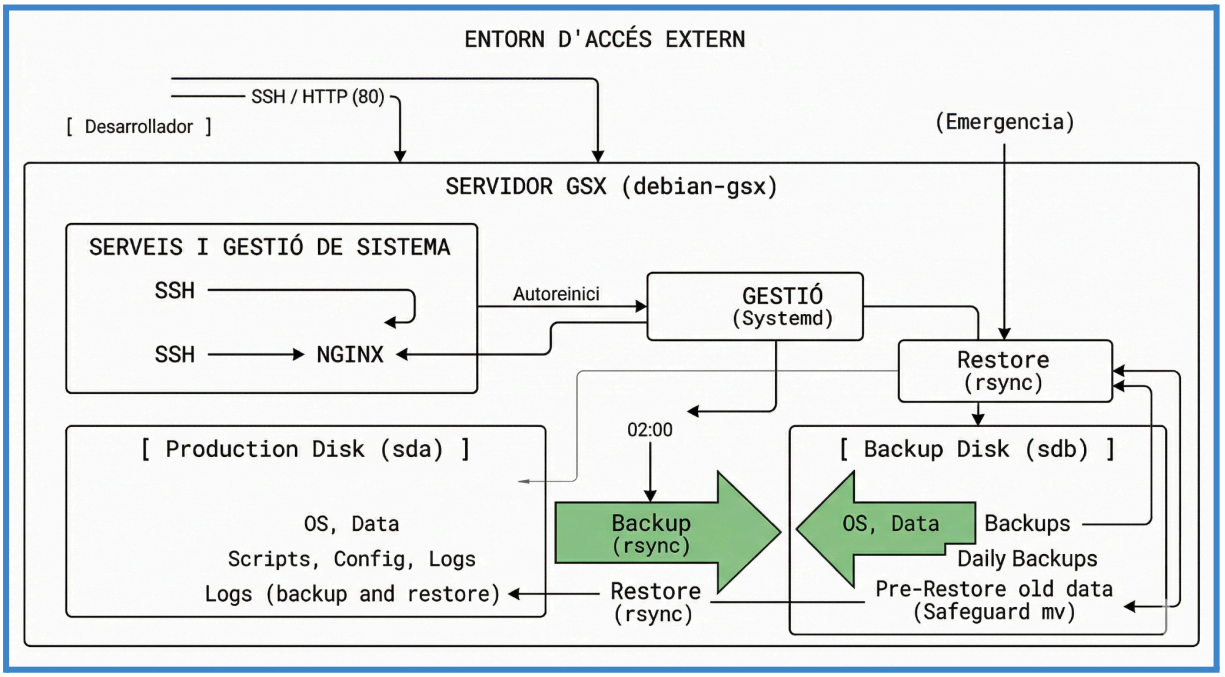
Si el servidor experimenta degradación de rendimiento, el ingeniero de guardia debe seguir estos pasos:

1. **Identificación rápida:** Ejecutar el script process_diag.sh o herramientas interactivas como htop para ordenar los procesos por consumo de %CPU o %MEM.
2. **Análisis de Árbol:** Ejecutar pstree -pnhu para ver qué servicio padre está generando los subprocesos que consumen recursos.
3. **Contención:** Si el proceso pertenece a un usuario o servicio fuera de control (y no es vital para el sistema), enviarle una señal de terminación elegante: kill -15 <PID>.

1.4. Procedimiento de Escalado

Si un problema crítico (como una caída total del disco o una intrusión) no puede ser resuelto en un plazo de 30 minutos utilizando este Runbook, el incidente debe ser escalado inmediatamente al Ingeniero de Sistemas Senior (en el contexto académico, avisar al profesor).

Arquitectura del Sistema




```
/admin/
├── backups/            # Directorio reservado para backups de los scripts.
├── config/             # Repositorio crítico de scripts operativos y archivos de configuración.
│   ├── backup_manager.sh    # Backup de la carpeta /home/greendevcorp/.
│   ├── backup.sh            # Backup de los scritps del admin.
│   ├── custom_logs.conf     # Configuración personalizada para la gestión de logs (Logrotate).
│   ├── restore_backup.sh    # Script de recuperación ante desastres (DRP).
│   └── verify_backup.sh     # Auditor de integridad y verificador de copias.
├── logs/               # Centro de monitorización y auditoría del sistema.
│   ├── backups/             # Subdirectorio para logs específicos de transferencias antiguas.
│   ├── greendev_backup.log  # Registro detallado de la última actividad de backup.
│   └── integrity_check.log  # Resultados de las pruebas de 'verify_backup.sh'.
└── scripts/                 # Directorio para utilidades de scripts personales del admin.

/home/
├── dev1/ , dev2/ ...        # Directorios personales (Homes) de cada desarrollador.
│   └── (vacío)
└── greendevcorp/            # Directorio Raíz del Grupo (Espacio Compartido).
    ├── bin/                 # Almacena scripts y binarios ejecutables propios del equipo.
    ├── done.log             # Archivo central de registro de tareas finalizadas por los desarrolladores.
    └── shared/              # Carpeta de intercambio con bit de permanencia (Sticky Bit) y SGID.

/mnt/data_storage/
├── backups_greendevcorp/    # Repositorio principal de copias de seguridad del equipo.
│   └── latest ->            # Enlace simbólico al backup más reciente.
└── pre_restore_old_data_.../  # Carpetas de salvaguarda creadas antes de una restauración.
```

2.1. Decisiones de Diseño (Design Rationale) y Compromisos (Trade-offs)

- **Automatización vs. Configuración Manual:** Se ha invertido tiempo en automatizar todo mediante *Systemd* y scripts en *Bash*. El compromiso (*trade-off*) es una mayor curva de aprendizaje y tiempo de desarrollo inicial, a cambio de una recuperación instantánea e idempotencia total frente a errores manuales.
- **Aislamiento de Discos:** Separar el disco de Sistema (sda) del disco de Datos/Backups (sdb) evita que un ataque de denegación de servicio que sature los

logs colapse los backups. El compromiso es la gestión de múltiples puntos de montaje en /etc/fstab.

2.2. Planificación Futura y Escalabilidad

El sistema actual es monolítico y está diseñado para una startup de 4 a 10 personas. Si la empresa escala a 20 o 100 empleados, esta arquitectura deberá evolucionar:

- **Gestión de Usuarios:** No sería viable usar comandos useradd locales. Se deberá implementar un servicio de directorio centralizado como **LDAP** o **Active Directory**.
- **Disponibilidad Web:** Un solo Nginx en una sola máquina virtual se convertiría en un cuello de botella. Habría que implementar un **Load Balancer** distribuyendo tráfico a múltiples nodos.
- **Infraestructura como Código (IaC):** Los scripts en Bash, aunque funcionales, deberían migrarse a herramientas declarativas como **Ansible** o **Terraform** para gestionar docenas de servidores simultáneamente.

Preguntas de reflexión (Ivan)

¿Cuál fue el aspecto más difícil de este proyecto?

Lo más complicado ha sido, sin duda, la Semana 5, específicamente la estructuración y gestión de las particiones. El proceso de diseñar el almacenamiento, montarlo de forma persistente y asegurar que los scripts de backup y verificación interactuaran correctamente con las rutas del disco de 5GB fue un reto técnico importante.

Tuve que trabajar de manera iterativa, deshaciendo y rehaciendo la partición varias veces para corregir errores de montaje o permisos, lo que me enseñó la importancia crítica de tener una base de almacenamiento sólida antes de automatizar cualquier servicio de respaldo.

¿Qué harías diferente si pudieras empezar de cero?

Aunque estoy muy satisfecho con los cimientos que pusimos al inicio, si empezara de cero con el conocimiento que tengo ahora, definiría con mucha más antelación y precisión la arquitectura de directorios que utilizarían los usuarios.

Habiendo completado ya el despliegue, veo que tener una estructura de carpetas (como /home/greendevcorp/bin o shared) totalmente clara desde el primer minuto ahorra mucho tiempo en la configuración posterior de permisos y variables de entorno. Una planificación previa más detallada de "quién accede a qué" habría hecho el proceso de scripting mucho más fluido.

¿Cómo ha cambiado tu visión de lo que es la administración de sistemas?

Mi visión ha evolucionado al descubrir el enorme potencial de personalización y automatización que ofrece Linux. Me ha sorprendido gratamente la "comodidad" que se puede alcanzar mediante el uso estratégico de servicios y timers de Systemd.

Antes veía la administración como una serie de tareas manuales, pero ahora entiendo que un buen administrador es aquel que diseña un sistema capaz de vigilarse y gestionarse a sí mismo (como los backups automáticos o los scripts de integridad), permitiendo que la infraestructura trabaje para el usuario y no al revés.

¿Qué es algo sobre lo que te gustaría aprender más?

Me he quedado con muchas ganas de profundizar en la seguridad del sistema, especialmente en el funcionamiento interno del sistema PAM (Pluggable Authentication Modules) para gestionar el acceso de forma más granular.

También tengo mucho interés en aprender sobre sistemas de archivos de red como NFS (Network File System), ya que me gustaría entender cómo aplicar las lógicas de backup y compartición de archivos que hemos visto en esta práctica, pero en un entorno distribuido donde los datos no están físicamente en la misma máquina que los usuarios.

Preguntas de reflexión (Sergi)

¿Cuál fue el aspecto más difícil de este proyecto?

El reto más complejo fue lograr la automatización total ("Zero-Touch Provisioning") sorteando los reinicios del sistema. Al principio, nuestro script de la Semana 1 cortaba la ejecución al hacer el *reboot*. Aprender a crear un servicio efímero oneshot en systemd que sobreviviera al reinicio, se autolimpiara y continuara ejecutando la Fase 2 en segundo plano (lidiando además con el cifrado desatendido de GPG sin consola interactiva) fue un dolor de cabeza, pero el resultado fue muy profesional.

¿Qué harías diferente si pudieras empezar de cero?

Estructuraría mucho mejor el código desde el día uno. Al principio escribíamos scripts pensando solo en que "funcionaran", pero luego nos dimos cuenta de que la *idempotencia* es clave (poder ejecutar un script 100 veces sin romper nada). Si empezara de nuevo, diseñaría la arquitectura de los directorios y los servicios en papel antes de picar una sola línea en Bash.

¿Cómo ha cambiado tu visión de lo que es la administración de sistemas?

Antes pensaba que un Sysadmin era simplemente la persona que tecleaba comandos en una terminal para arreglar servidores rotos o crear usuarios. Ahora entiendo que la administración moderna es básicamente ingeniería de software aplicada a la infraestructura : se trata de escribir código (Infraestructura como Código), prever fallos, garantizar la observabilidad con logs y asegurar que el sistema se recupere solo.

¿Qué es algo sobre lo que te gustaría aprender más?

Me he dado cuenta de que, aunque Bash es muy potente para 1 o 2 servidores, sería inmanejable para 100. Me gustaría aprender a fondo herramientas declarativas de

automatización a gran escala como Ansible o Terraform, y también me he quedado con ganas de explorar la orquestación con contenedores (Docker/Kubernetes) para no depender tanto de la máquina virtual completa.